CS 630: Software Testing and Quality Assurance

7-2 Paper: Quality Management Plan

Malgorzata Debska

Southern New Hampshire University

Dr. Ben Torrales

August 20, 2026

Quality Management Plan for LogiCo

LogiCo relies on its software platform for user account management, route optimization, package tracking, billing, and third-party integrations. The system combines a Java-based monolithic application with Node.js and Python microservices that still share an Oracle database. This structure creates dependencies that make changes difficult to coordinate and test. LogiCo also has no dedicated QA team, and testing practices vary among development teams. This quality management plan establishes measurable standards, QA activities, responsibilities, and monitoring methods that support reliable software delivery without unnecessarily slowing development.

Quality Standards and Benchmarks

LogiCo's main goals are to reduce production defects, decrease delivery time, improve schedule and quality predictability, and increase customer satisfaction. However, leadership has not yet established measurable benchmarks for these goals. Therefore, the first program increment should be used to establish baseline measurements. After establishing the baseline, LogiCo should target a 25% reduction in customer-reported production defects within two program increments. Critical defects that prevent customers from completing important tasks should receive the highest priority. LogiCo should also aim for at least 80% automated test coverage of critical business workflows, including billing, authentication, package tracking, and route optimization. For normal releases, at least 95% of required CI/CD quality checks should pass before production deployment and required pipeline steps should not be bypassed. This standard addresses the current problem of developers occasionally skipping Jenkins steps when under

time pressure. Together, these benchmarks give teams measurable expectations for determining whether software quality is improving.

QA Activities Across the Development Life Cycle

QA activities should begin during planning rather than being delayed until the hardening sprint. During sprint planning, developers and product owners should establish acceptance criteria and identify risks, dependencies, and testing requirements for each user's story. High-risk features, especially billing and changes affecting shared APIs or databases, should receive additional review. During development, developers should create unit tests for new functionality and run them before submitting pull requests. Peer reviews should verify both code quality and associated tests. Jenkins should continue running automated unit and API tests while LogiCo gradually adds integration, contract, and regression tests. These additions are important because integration and contract testing are currently rare, and there is no consistent regression-testing approach. Before release, automated regression and exploratory testing should be completed in staging. Contract testing should confirm that API changes do not unexpectedly affect mobile applications or other services, which is particularly important because LogiCo’s APIs are not versioned. After deployment, teams should monitor production errors and customer-reported defects. Serious escaped defects should receive a root-cause review to determine how similar problems can be prevented.

QA Roles and Responsibilities

Because LogiCo does not currently employ dedicated QA personnel, quality should be treated as a shared responsibility. Developers should perform unit testing, maintain automated tests,

participate in peer reviews, and correct defects. Product owners should establish acceptance criteria and verify that completed functionality meets business requirements. Technical leads should coordinate integration and contract testing when changes affect multiple services. DevOps personnel should maintain Jenkins, Docker deployments, staging environments, and automated quality gates. Mobile representatives should participate in reviews involving APIs because mobile releases depend on stable back-end services and follow stricter release schedules. Each development team should also designate a QA champion. The QA champion would help enforce common standards, review quality metrics, identify testing gaps, and coordinate quality concerns with other teams. This approach provides accountability without requiring LogiCo to immediately establish a separate QA department.

Tracking Quality Progress

LogiCo should maintain a shared quality dashboard that provides teams and leadership with consistent information. Jenkins can track automated test results, build failures, and pipeline performance, while the defect-tracking system should consistently record defect severity, affected component, and root cause. Key metrics should include production defect count, critical defects, defect escape rate, automated test pass rate, regression failures, deployment frequency, and lead time changes. LogiCo should also monitor staging availability because the environment currently experiences approximately 10% downtime, which delays validation and reduces confidence in testing. Metrics should be reviewed at the end of each sprint, with a broader evaluation at the end of every program increment. Results should be compared with the original baseline so that teams can identify trends and take corrective action when quality declines.

Alignment With Strategic Goals

Each part of the QMP supports LogiCo's organizational priorities. Earlier and more consistent testing should reduce customer-facing defects and improve customer satisfaction. Automated testing and CI/CD quality gates provide faster feedback, reduce rework, and support shorter delivery times. Shared standards and metrics also make release of quality and schedules more predictable. The plan is designed around LogiCo's practical limitations. Leadership recognizes that teams are more likely to support changes that reduce rework and increase confidence without significantly slowing delivery. Integrating quality practices into existing development activities allows LogiCo to strengthen QA without introducing excessive manual processes.

To conclude LogiCo's quality problems result largely from inconsistent testing, limited coordination, unclear standards, and unstable development practices. Establishing measurable benchmarks, continuous QA activities, clear responsibilities, automated quality gates, and regular progress reviews will create a more consistent approach to software quality. This QMP will help LogiCo reduce production defects, improve release predictability, maintain development efficiency, and provide customers with a more reliable software experience.